

Remaining Work & Setup Guide

Purpose: what's built, what's partially built, and exactly what needs **your manual action** to make the platform fully operational. Written for the state of the platform as of this session.

Legend:  done ·  partial (works, needs config/data) ·  not started ·  needs your manual action

0. TL;DR — the manual actions, shortest path

#	Action	Why
1	Run the backend with <code>./start-backend.sh</code> (not <code>uvicorn --reload</code>)	<code>--reload</code> restarts the server every time a run writes a file — kills runs.
2	Add the <code>.env</code> values in §1 (Slack, Jira, CI secret, PR device, autotest)	Turns on notifications, tickets, and the honest PR path.
3	Record + tag scenarios for each app (Business has 0 today)	Nothing is actually tested without scenarios → PRs show "no tests".
4	Give me the staging repo URL + bundle id	To wire the staging environment + its Metro port.
5	For CI: register a self-hosted macOS runner + copy the workflow + set repo secrets	Real per-PR CI (iOS sims can't run on GitHub-hosted runners).
6	Confirm with Tanish the bot reads <code>PLATFORM_SCENARIO_CALLBACK</code> each run	So the 70-scenario suite reports into the dashboard.

1. Configuration you must set (`.env`)

Add these to `.env` at the project root (all optional — each feature no-ops until set):

```
# --- PR auto-test ---
PR_AUTOTEST_ENABLED=true                # poller runs the HONEST smart-
selected autotest                        # (plan → build → run → comment)

instead of a build-only pass
PR_TEST_IOS_DEVICE=DA24A392-...         # pin PR runs to a specific
simulator (your iPhone 16 Pro)

# --- Notifications (Slack) ---
SLACK_WEBHOOK_URL=https://hooks.slack.com/services/XXX/YYYY/ZZZ

# --- Jira auto-tickets ---
JIRA_URL=https://yourteam.atlassian.net
JIRA_EMAIL=you@company.com
JIRA_TOKEN=<atlassian-api-token>
JIRA_PROJECT_KEY=QA
```

```
# --- CI trigger ---
CI_SECRET=<any-long-random-string>      # must match the GitHub secret of
the same name

# --- Optional ---
DASHBOARD_URL=http://localhost:5173      # used in Slack/Jira links
LLM_MODEL_NAME=llama3.1:8b               # better report/planner quality
than the default llama3.2
```

Restart the backend (`./start-backend.sh`) after editing `.env`.

2. CI / CD integration 🟡👤

Built ✅

- GitHub **webhook** endpoint (auto-queues on PR/push).
- **PR poller** — checks each project's open PRs every 5 min and auto-tests new commits.
- **Autotest** flow — plan → smart-select scenarios (dependency graph) → build PR branch → run → comment verdict on the PR → Slack/Jira.
- **CI endpoint** `POST /api/v1/ci/pr-autotest` (secret-guarded).
- A **real GitHub Actions workflow** template: `automation/ci_cd/github_actions.yml`.

Needs your manual work 👤

1. **Self-hosted macOS runner** — iOS simulators can't run on GitHub-hosted runners. On each app repo: *Settings* → *Actions* → *Runners* → *New self-hosted runner (macOS)*, install it on the Mac that runs the platform.
2. Copy `automation/ci_cd/github_actions.yml` → `.github/workflows/pr-ios-qa.yml` in the **app repo** (vya-consumer / vya-business).
3. Set app-repo secrets: `PLATFORM_URL` (e.g. `http://localhost:8000`) and `CI_SECRET` (matching `.env`).
4. Set `CI_SECRET` and `PR_AUTOTEST_ENABLED=true` in the platform `.env`.

Alternative (no self-hosted runner): the **PR poller already works** — enable `PR_AUTOTEST_ENABLED=true` and it auto-tests + comments without any GitHub Actions setup. The Actions workflow is only needed if you want the check to appear as a PR status check.

Not done 🛑 : Jenkins / Azure DevOps pipelines (only YAML stubs exist); a hosted/public platform URL (needed if the runner is not on the platform Mac).

3. Tanish's Vya bot (70-scenario suite) 🟡👤

Built ✅ (platform side)

- `/api/v1/runs/{run_id}/scenario-result` — receives per-scenario results, **upserts one row per (run, scenario)**, merges the bot's separate Consumer + Business posts into one row with the **both-must-pass** gate.

- `automation/android_bot/run_vya_bot_suite.py` — creates a platform run, wires `PLATFORM_SCENARIO_CALLBACK`, and launches the bot for a selection (`all` | `CW1`, `PAY8` | `1-10`).
- The bot is already **generic** (reads the callback URL from an env var — no hardcoding).

Needs manual work

1. **Confirm with Tanish** that `shared/scenario_reporter.py` reads `PLATFORM_SCENARIO_CALLBACK` (or the callback URL) **on every run** and POSTs each result. See `automation/android_bot/TANISH_BOT_PATCH.md`.
2. The bot runs on the **Mac** and needs its device setup (`idb` + the simulators/devices it drives). This is the bot's own environment, not the platform's.
3. To point the bot at this platform, set `PLATFORM_BASE_URL` when launching the suite (defaults to `http://localhost:8000`).

Status: the wiring is complete; it needs the bot's device setup on your side + Tanish's confirmation, then the ~70 scenarios stream straight into the Scenarios tab.

4. Scenario libraries — the biggest lever

Built

- **Recorder** (mirror the sim, click to capture — records real testIDs).
- Scenario **editor / CRUD, Run, Run All** (one Appium session), live results, **self-healing** locators.
- **Coverage tags** + one-click  **Suggest** (maps steps → the app's real screens).
- **5 scenarios** for the **Consumer app**.

Needs manual work

- **The Business app has 0 scenarios** — that's why its PRs show "no tests" (an honest verdict, not a pass). Record its key flows (login → waiter accept → kitchen serve → bill).
- **Record the real flows per app** and **click Suggest** to tag coverage — smart PR selection only skips tests for **tagged** scenarios; untagged ones always run.
- Re-record any scenario made before the recorder's locator fix so it carries reliable testIDs.

Why this matters: everything downstream (smart selection, autotest, reports, the "% reduction") is only as good as the tagged scenario library. This is the main ongoing manual work.

5. Environments — prod vs staging

Built

- An **environment = a project**; a scenario runs against a chosen environment with **"Pull latest & rebuild"** (handles staging's daily builds).
- Per-app **Metro port routing** (Business = 8082) + `RCT_jsLocation`.

Needs manual work / from you

1. **Give me the staging repo git URL (+ branch) and its bundle id.** I'll add it as a "Vya Staging" project and pin its Metro port so prod + staging don't collide.

2. Add **staging credentials** if they differ from prod.
3. Then: run any scenario → pick environment **Vya Staging** → "Pull latest & rebuild".

6. Notifications & reporting

Built

- **Slack** message on every PR-QA verdict; **Jira** auto-ticket on failure (with RCA); manual **"File Jira"** button on any report.
- Reports tab: AI narrative (Ollama), **HTML + PDF** export, **trends** (pass-rate over time, per-environment, flaky), and an **honest verdict** ("no tests" for 0-scenario runs — never a false green).

Needs manual work : set the Slack/Jira env values (§1). Optionally `LLM_MODEL_NAME=llama3.1:8b` for better-written reports (`ollama pull llama3.1:8b`).

7. Infrastructure / deployment

Not started — decisions needed

- **Database:** currently **SQLite** (`test_automation_new.db`). For multi-user/hosted use, migrate to **Postgres** (set `DATABASE_URL`). Redis is referenced but not required.
- **Deployment:** `docker-compose.yml` exists but the platform is **not containerized/hosted** — it runs on your Mac. To host the **dashboard + API** (the Render thread), you'd: deploy the FastAPI app publicly, make the dashboard's API base an **env var** (it's currently hardcoded to `http://localhost:8000`), and switch to Postgres. **The bot/Appium/sims must stay on a Mac** regardless.
- **Live streaming** uses frame **polling**, not WebSockets (works; WS would be smoother).

Recommendation: leave infra last — it only matters when you move off the Mac.

8. Operational notes

- **Run the backend with** `./start-backend.sh` (no `--reload`). `--reload` watches the whole project and the platform writes files during runs (DB, screenshots, builds, graph cache) → it would restart mid-run and kill everything. For code development, use the scoped form in the script's comments.
- **Appium self-heals:** if WebDriverAgent wedges ("device not working"), a run now clears and restarts it automatically. The Scenarios tab shows an **engine-status** chip (ready / will auto-start).
- **First run after any restart** rebuilds WebDriverAgent (~20–30s at "Connecting...") — this is expected, not a hang; the live log says so.

9. Feature status at a glance

Area	Status	Your action
iOS execution (XCUITest/WDA)		—

Area	Status	Your action
Scenario recorder	✓	Record flows
Scenario run / Run All / self-healing	✓	—
Coverage tags + AI suggest	✓	Click Suggest / tag
Git change analyzer	✓	—
Graph-driven smart test selection	✓	Tag scenarios
PR planner (what to test / how to reach)	✓	—
Autonomous PR loop (build → run → comment)	✓	PR_AUTOTEST_ENABLED=true
RCA + reports + trends + PDF/HTML	✓	—
Honest verdicts (no false green)	✓	—
Slack notifications	●	Set SLACK_WEBHOOK_URL
Jira auto-tickets	●	Set JIRA_*
CI (GitHub Actions)	●	Self-hosted runner + secrets
Environments (prod/staging)	●	Give staging repo + bundle id
Vya bot (70 scenarios)	●	Bot device setup + Tanish confirm
Scenario library per app	●	Record + tag (Business = 0)
Postgres / Docker / hosting	🚫	Decide when hosting
Jenkins / Azure pipelines	🚫	Only if needed

Ask me to do any of the ✓/● platform-side wiring; the 👤 items are the ones that need you (credentials, recording flows, registering runners, the staging repo).